Приложение. Кейс 3. Онлайн-чаевые

Кейс-чемпионат «ВПОТОКЕ», команда НаЧай: Несмачный Егор Дмитриевич (аналитик), Кожаев Тимур Владимирович (архитектор), Дурнов Егор Кириллович (разработчик), Солнышкин Леонид Андреевич (презентатор). Версия от 01.11.2026.

Сеть ресторанов «Вкус-Сервис» хочет внедрить онлайн-чаевые: клиент сканирует QR на чеке, выбирает размер чаевых и платит. Платформа хранит мерчантов, официантов и платежи и формирует отчётность. На российском рынке такие сервисы уже есть, а в Юго-Восточной Азии ниша свободна.

Код прототипа, тесты и заполненные шаблоны: github.com/brokentwo/online-tips-case-3.

Что не сходится в материалах кейса

Мы проверили материалы скриптом и нашли противоречия. Решения легко изменить после ответа организаторов.

- **Формула и тесты.** Формула `md5(qr_id + nonce) % 10` даёт qr_001 код 9 (таймаут), а qr_003 код 7 (успех), но Т3-1 ждёт SUCCESS, а Т3-2 ждёт DECLINED. Мы закрепили эти два QR как контрольные, ответ показывает это полем `control_qr`. Остальные 98 QR считаются по формуле.
- **История платежей.** Строка p_0003 с суммами из Т3-2 имеет статус SUCCESS. `payments.csv` считаем историей: его статусы идут по номеру строки и с формулой не связаны.
- **Даты.** Все `exp` приходятся на 1–4 ноября 2025, по настоящим часам все QR просрочены. Часы прототипа стоят на `2025-11-01T18:00:00Z`, это время из шаблона.
- **Файлы.** В `08_errors.md` лежит шаблон ER-модели, каталога ошибок и `10_tests.md` нет. Каталог собран из подсказок в шаблонах, наши коды в нём отмечены.
- **Отчёт.** В образце `tips_total = sum(tip)` без учёта статуса. Деньги считаем только по SUCCESS: для m_001 это 414,85 ₽ вместо 611,20 ₽.
- **Слайды.** В материалах три разные структуры. Порядок взят из задания, титул, схема и Git добавлены из общей инструкции.

A. Диаграммы

Исходники схем для draw.io лежат в папке `схемы` рядом с кодом.

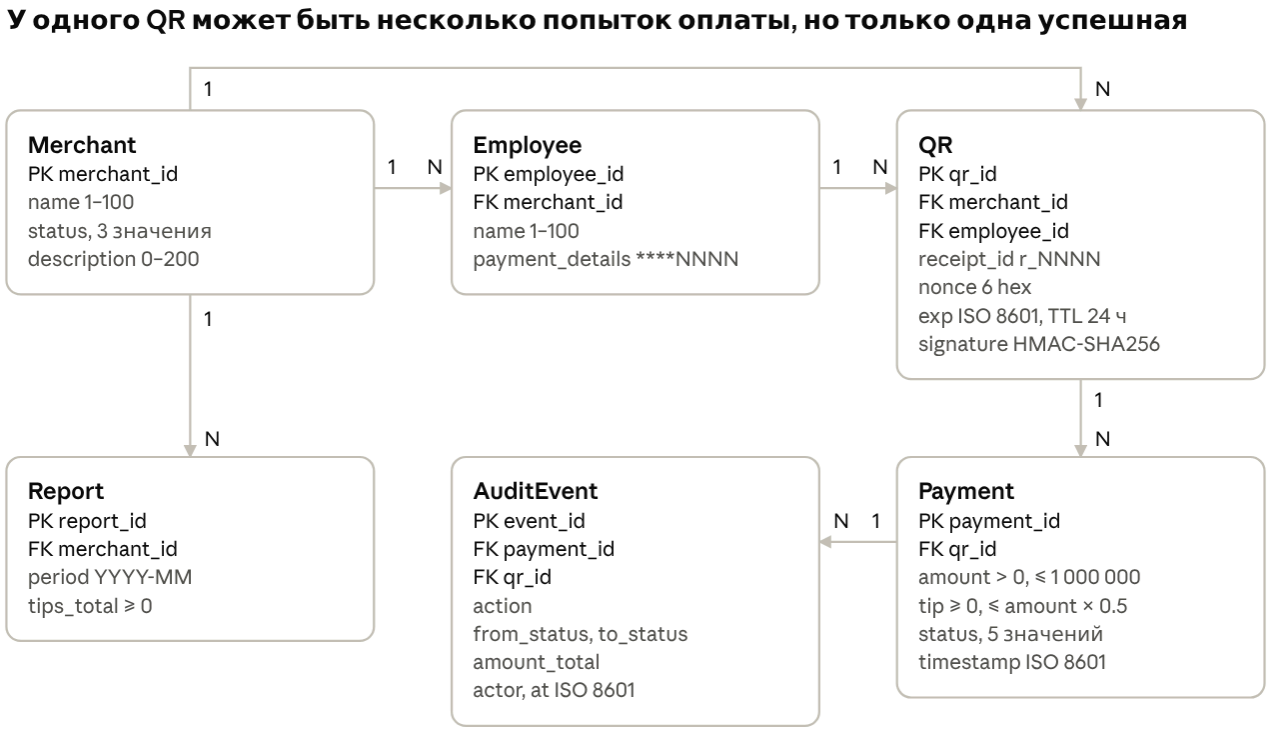
A.1. Схема системы



схема системы · 6 блоков, потоки данных подписаны

- **Клиент:** телефон без регистрации. Передаёт QR-код, выбор чаевых и подтверждение.
- **Мерчант, официант:** кабинет заведения. Передают запрос QR на чек и запрос отчёта.
- **Платформа:** сервис чаевых. Отдаёт форму и результат клиенту, запрос эмулятору, строку платежа хранилищу, событие аудиту.
- **Эмулятор шлюза:** замена банка. Возвращает код 0..7 SUCCESS, 8 DECLINED, 9 TIMEOUT.
- **Хранилище:** CSV в прототипе, БД в боевой версии. Хранит справочники, QR и платежи.
- **Аудит:** журнал `audit_out.csv`. Принимает событие из 9 полей без реквизитов и имён.

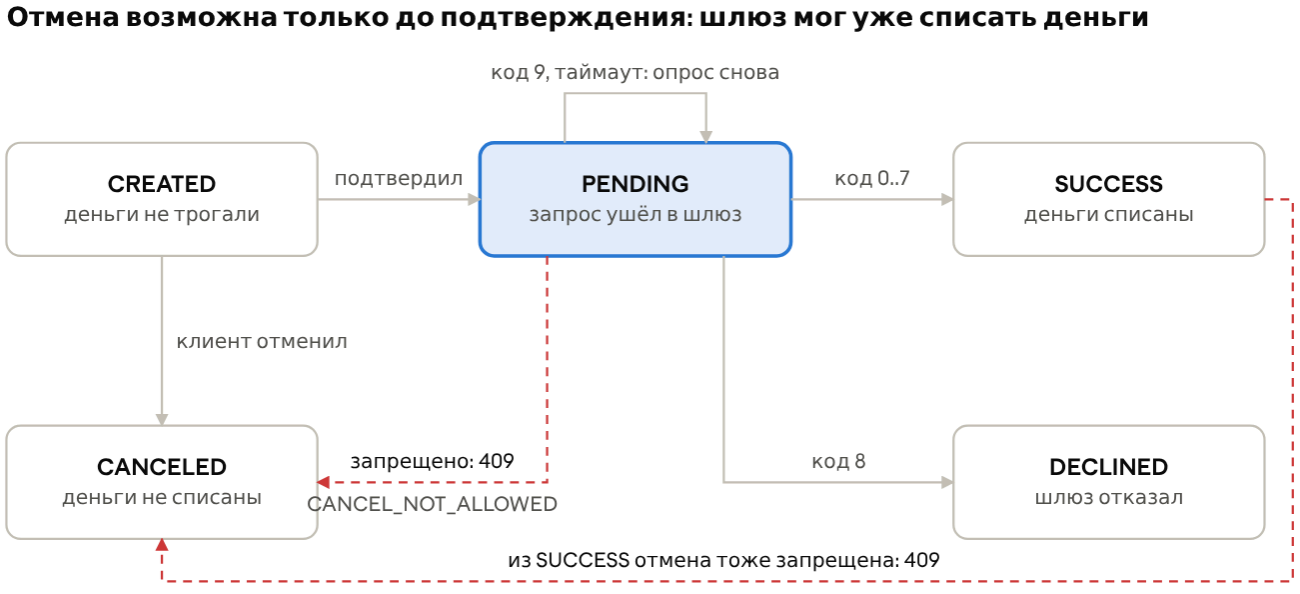
A.2. ER-модель



ER-модель · 6 сущностей, PK, FK и связи 1:N

В `payments.csv` вместо `qr_id` лежат `merchant_id` и `employee_id`, поэтому связь с QR прототип хранит в `payment_index.csv`. Report считается по запросу и не хранится.

A.3. Схема состояний платежа



состояния платежа · 5 статусов, 2 запрещённых перехода

Статус меняют клиент (подтверждение, отмена) и эмулятор (код ответа). Повторный опрос после таймаута считает код с номером попытки, иначе ответ никогда бы не изменился.

B. Таблицы

B.1. Правила QR и платежа

Правило	Код ошибки	Обоснование
Q-1. Подпись совпадает с HMAC	`INVALID_QR`	Защита от подделки QR
Q-2. `exp` не раньше текущего времени	`QR_EXPIRED`	Старый чек нельзя оплатить позже
Q-3. `nonce` ещё не занят	`QR_USED`	Один чек нельзя оплатить дважды
Q-4. Мерчант в статусе `ACTIVE`	`MERCHANT_BLOCKED`	Заблокированное заведение не принимает деньги
Q-5. `receipt_id` совпадает с QR	`RECEIPT_MISMATCH`	QR работает только для своего чека
Q-6. `tip_pct` от 0 до 50	`TIP_OUT_OF_RANGE`	Ловит ошибку ввода и завышение чаевых
P-1. `base_amount` больше 0 и не больше 1 000 000	`AMOUNT_OUT_OF_RANGE`	Защита от неверной суммы
P-2. Повтор `idempotency_key`: тот же платёж, HTTP 200	нет ошибки	Повторное нажатие не списывает деньги второй раз
P-3. `amount_total` не больше 2 000 000	`AMOUNT_OUT_OF_RANGE`	Страховка: при P-1 и Q-6 итог не выше 1 500 000
P-4. Отмена из `PENDING` запрещена	`CANCEL_NOT_ALLOWED`	Шлюз мог уже списать деньги

Порядок проверок: Q-1, P-2, Q-2, Q-3, Q-4, Q-5, P-1, Q-6, P-3. Подпись идёт первой, чтобы чужой платёж нельзя было получить по ключу повтора. P-2 идёт раньше срока и nonce, иначе повтор получил бы ошибку на свой же платёж. Nonce занят,

пока платёж в `CREATED`, `PENDING` или `SUCCESS`: после отказа и отмены возможна новая попытка.

B.2. Матрица «ситуация → код ошибки → HTTP → retryable»

Ситуация	Код ошибки	HTTP	retryable
Подпись QR не совпала	`INVALID_QR`	422	нет
QR истёк	`QR_EXPIRED`	410	да (новый QR)
nonce использован	`QR_USED`	409	нет
merchant BLOCKED	`MERCHANT_BLOCKED`	403	нет
Чек не совпадает с QR	`RECEIPT_MISMATCH`	422	нет
Чаевые больше 50%	`TIP_OUT_OF_RANGE`	422	да (другой процент)
Чек вне пределов или итог больше 2 000 000	`AMOUNT_OUT_OF_RANGE`	422	нет
Тот же ключ и те же данные	нет ошибки, тот же платёж	200	не нужен
Тот же ключ и другие данные	`IDEMPOTENCY_CONFLICT`	409	нет
Эмулятор вернул 8	`CARD_DECLINED`	402	да (новая попытка)
Эмулятор вернул 9	`GATEWAY_TIMEOUT`	200, статус `PENDING`	да (повторный опрос)
Отмена из `PENDING` или `SUCCESS`	`CANCEL_NOT_ALLOWED`	409	нет
Чужая роль или чужой токен	`FORBIDDEN`, `BAD_CONFIRM_TOKEN`	403	нет

Полный каталог с текстами для пользователя лежит в `08_errors.md`.

B.3. Тесты

Факт записан из запуска `python3 tests_t3.py` от 01.11.2026 на одном Mac. T3-1… T3-5 заданы организаторами. T3-6 мы добавили, потому что общая инструкция требует сценарий «Таймаут/STALE»; STALE в этом кейсе — истёкший QR из T3-3.

Тест и вход	Ожидание	Факт	Пройден
T3-1. Успех: qr_001, base=637.00, tip_pct=5	SUCCESS, amount_total=668.85, tip=31.85	HTTP 200, SUCCESS, 668.85 и 31.85; суммы совпали с p_0001	да
T3-2. Отказ шлюза: qr_003, base=911.00, tip_pct=15	DECLINED, CARD_DECLINED	HTTP 402, DECLINED, CARD_DECLINED; строка записана	да
T3-3. Неверный QR: испорченная подпись, подмена официанта, истёкший срок	INVALID_QR (422) или QR_EXPIRED (410), платёж не создан	422 INVALID_QR дважды, 410 QR_EXPIRED; строк в файле 0, отказов в журнале 3	да
T3-4. Повтор: тот же idempotency_key дважды	Тот же payment_id, одна строка	p_0101 в обоих ответах, HTTP 200; строк 1, списаний 1; другая сумма с тем же ключом: 409 IDEMPOTENCY_CONFLICT	да
T3-5. Отмена: до подтверждения и из PENDING	CANCELED; из PENDING 409	Из CREATED: 200 CANCELED; из PENDING и SUCCESS: 409 CANCEL_NOT_ALLOWED	да
T3-6. Таймаут: qr_010, код 9, повторный опрос	PENDING, GATEWAY_TIMEOUT, retryable; один платёж	HTTP 200, PENDING, GATEWAY_TIMEOUT, retryable; повторный опрос: DECLINED; строк 1	да

Ещё 17 проверок: Q-3…Q-6, P-1, P-3, чужой токен, подпись всех 100 QR, суммы всех 100 платежей, формула эмулятора, генерация QR, отчёты, права, журнал, маска реквизитов, скорость подписи и неизменность исходных файлов. Все пройдены, факты лежат в `test_results.md`.

Чтобы проверить сами тесты, `tests_mutation.py` намеренно ломает код в 37 местах. Тесты заметили все 37 поломок.

B.4. Роли и регистрация

Действие	Мерчант	Официант	Клиент	Администратор
Регистрация	да	по приглашению	нет	нет
Генерация QR на чек	да	свой чек	нет	нет
Свои чаевые	да	только свои	нет	нет
Сканирование QR	нет	нет	свой чек	нет
Оплата чаевых	нет	нет	да	нет
Отчёт по заведению	своё	нет	нет	все
Блокировка мерчанта	нет	нет	нет	да
Справочники	свои сотрудники	нет	нет	да

Регистрация мерчанта

1. Мерчант подаёт заявку: название, описание, контакт. Проверки: `name` 1–100, `description` до 200, заявки ещё нет.
2. Система создаёт запись `m_NNN` со статусом `PENDING` и пишет в журнал. При `PENDING` QR и оплата запрещены.
3. Администратор активирует или отклоняет заявку с причиной. Мерчант не может активировать себя сам.
4. Система открывает кабинет и выпуск QR и пишет в журнал. Разрешён только переход `PENDING` → `ACTIVE`, ключ подписи не выдаётся.

Регистрация сотрудника

1. Мерчант приглашает сотрудника: система создаёт запись `e_NNN` и одноразовую ссылку на 24 часа.
2. Официант подтверждает приглашение и так привязывается к одному мерчанту.
3. Реквизиты вводятся у провайдера выплат, в Платформу попадает только маска `****1234`.
4. Система активирует сотрудника и пишет событие в журнал без реквизитов.

B.5. Требования

Функциональное требование	Приоритет	Тест
FR-1. Сгенерировать QR на чек	Must	T3-1, проверка FR-1
FR-2. Проверить подпись и срок QR	Must	T3-3
FR-3. Показать форму чаевых	Must	проверка «Маска»
FR-4. Рассчитать чаевые и итог	Must	T3-1, Q-6
FR-5. Создать платёж без дублей	Must	T3-4
FR-6. Эмулировать шлюз	Must	T3-1, T3-2, T3-6
FR-7. Отменить платёж до подтверждения	Must	T3-5
FR-8. Записать платёж и переходы статуса	Must	T3-4, проверка «Аудит»
FR-9. Отчёты: мерчант, официант, дни	Must	проверка «Отчёты»
FR-10. Регистрация мерчанта и сотрудника	Should	в прототипе нет

Нефункциональное требование	Результат
NFR-1. Подпись QR проверяется за 50 мс или быстрее	Меньше 0,01 мс на одной машине
NFR-2. TTL QR 24 часа, nonce одноразовый	Проверки FR-1, Q-3, T3-3
NFR-3. HMAC-SHA256, сравнение за постоянное время, ключ в `os.environ`	Проверка подписи всех QR
NFR-4. `payment_details` только маской `****NNNN`	Проверки «Маска» и «Аудит»
NFR-5. Не больше 5 неверных QR в минуту с адреса, затем 429	Только описано: в CLI нет адреса
NFR-6. Каждый переход и отказ QR попадают в журнал	Проверка «Аудит», T3-3

Сценарий 1. Клиент оставил чаевые. Официант выпускает QR и приносит чек. Клиент видит заведение, официанта и сумму, выбирает 5%, видит итог 668,85 ₽ и подтверждает. Клиент получает «Спасибо!», официант видит чаевые в своём отчёте.

Сценарий 2. Клиент отменил платёж. На экране подтверждения клиент передумал: статус `CANCELED`, деньги не списаны, QR снова работает. После подтверждения отмена запрещена: 409.

Сценарий 3. QR устарел. Клиент получает 410 `QR_EXPIRED`, официант выпускает новый QR на чек. Это разрешено, только если старый истёк и не оплачен.

B.6. Словарь данных

Четыре файла, 22 колонки. Полная таблица с типами и примерами лежит в 03_data_dictionary.md.

- **merchants, 20 строк:** `merchant_id` m_NNN; `name` 1–100 символов; `status` ACTIVE, BLOCKED или PENDING, ошибка `MERCHANT_BLOCKED`; `description` до 200 символов.
- **employees, 60 строк:** `employee_id` e_NNN; `merchant_id` есть в merchants, ошибка `EMPLOYEE_MISMATCH`; `name` 1–100 символов; `payment_details` только маска, пример `****1001`.
- **payments, 100 строк:** `payment_id` p_NNNN; `merchant_id`; `employee_id`; `amount` больше 0 и до 1 000 000, ошибка `AMOUNT_OUT_OF_RANGE`; `tip` от 0 до половины `amount`, ошибка `TIP_OUT_OF_RANGE`; `status` из 5 значений; `timestamp` ISO 8601.
- **qr_links, 100 строк:** `qr_id` qr_NNN, ошибка `QR_NOT_FOUND`; `merchant_id` и `employee_id` входят в подпись; `receipt_id` r_NNNN, уникален для мерчанта, ошибка `RECEIPT_MISMATCH`; `nonce` 6 hex, ошибка `QR_USED`; `exp` ISO 8601, ошибка `QR_EXPIRED`; `signature` 16 hex, ошибка `INVALID_QR`.

Проверка данных: 18 мерчантов `ACTIVE`, m_019 `BLOCKED`, m_020 `PENDING`; подпись верна у 100 из 100 QR; статусы платежей: `SUCCESS` 70, `DECLINED` 10, `PENDING` 10, `CANCELED` 10; чаевые точно 5, 10 или 15% от чека.

B.7. QR и диалоговая форма

```
payload = merchant_id|employee_id|receipt_id|nonce|exp
signature = HMAC-SHA256(SECRET, payload), первые 16 hex
m_001|e_001|r_1001|c82561|2025-11-01T23:59:00Z → 68d0f361ac38c12b
```

Генерация: мерчант или официант запрашивает QR на чек, сервер создаёт `qr_id`, `nonce`, `exp` и подпись, сохраняет запись и возвращает QR-код со ссылкой `https://tips.example/pay?qr={qr_id}&sig={signature}`. В шаблоне

ссылка содержит только `qr_id`, но номера идут подряд, и чужой чек можно открыть перебором, поэтому мы добавили подпись. Проверки: мерчант `ACTIVE`, сотрудник этого мерчанта, чек ещё без действующего QR.

1. **Приветствие:** заведение, имя официанта, сумма чека. Кнопка «Продолжить», `GET /api/v1/qr/{qr_id}`.
2. **Выбор чаевых:** 0, 5, 10, 15, 20% или своя сумма, итог. Кнопка «Оплатить», `POST /api/v1/tips`.
3. **Подтверждение:** чек, чаевые, итог. Кнопка «Подтвердить оплату», `POST …/confirm`, или отмена, `POST …/cancel`.
4. **Результат:** «Спасибо! Чаевые отправлены», «Оплата не прошла, попробуйте другой способ» или «Обрабатываем, проверьте через минуту».

B.8. Эмуляция платежа

1. Валидация: есть QR, сумма, процент или своя сумма. Ошибка: 422 `BAD_REQUEST`.
2. Проверка QR: Q-1, P-2, Q-2, Q-3, Q-4, Q-5. Ошибка: код правила, отказы подписи и срока пишутся в журнал.
3. Расчёт суммы: P-1, Q-6, P-3. Ошибка: 422, платёж не создан.
4. Создание: новый `p_NNNN`, статус `CREATED`. Запись идёт через временный файл, при сбое старый файл цел.
5. Подтверждение: верный `confirm_token`, статус `CREATED` или `PENDING`. Ошибка: 403 или 409.
6. Результат: код эмулятора. При отказе 402, при таймауте `PENDING`, и клиент проверяет позже.
7. Отмена: верный токен, статус `CREATED`. Ошибка: 409 `CANCEL_NOT_ALLOWED`.
8. Аудит: каждый переход и отказ QR пишется сразу, без реквизитов и имён.

```
tip_amount = round(base_amount × tip_pct / 100, 2), половина копейки вверх
amount_total = base_amount + tip_amount
h = int(md5(qr_id + nonce), 16) mod 10: 0..7 SUCCESS, 8 DECLINED, 9 TIMEOUT
```

Пример	Чек	%	Чаевые	Итог	h по формуле	Статус
qr_001	637.00	5	31.85	668.85	9, закреплён код 0	SUCCESS
qr_003	911.00	15	136.65	1047.65	7, закреплён код 8	DECLINED
qr_002	774.00	10	77.40	851.40	3	SUCCESS

Идемпотентность: форма создаёт ключ при открытии и повторяет его при каждой отправке. Тот же ключ и те же данные дают тот же `payment_id`, HTTP 200 и одну строку в файле, шлюз второй раз не вызывается. Поля аудита: `event_id`, `payment_id`, `qr_id`, `action`, `from_status`, `to_status`, `amount_total`, `actor`, `at`.

B.9. Отчётность

Числа посчитаны прототипом по `payments.csv` за ноябрь 2025. Деньги считаем только по `SUCCESS`: при отказе и отмене чаевых никто не получил.

- **Мерчант m_001,** `GET /api/v1/reports/tips?merchant_id=m_001&from=…&to=…`: `tips_total` 414.85, `payments_count` 6, `avg_tip_pct` 9.0, `success_rate` 0.83.
- **Официант e_001,** `GET /api/v1/reports/tips?employee_id=e_001&from=…&to=…`: `tips_total` 83.60, `payments_count` 2, `avg_tip_amount` 41.80, `top_day` 2025-11-03.
- **m_001 по дням,** `GET /api/v1/reports/tips/daily?merchant_id=m_001&from=…&to=…`: 01.11 это 245.90, 3 платежа, 10.0%; 03.11 это 168.95, 3 платежа, 7.5%.

Формулы: `tips_total = sum(tip)` по `SUCCESS`, `avg_tip_pct = avg(tip / amount × 100)` по `SUCCESS`, `success_rate = count(SUCCESS) / count(*)`. Отчёт официанта видят он сам, его мерчант и администратор, коллегам ответ 403. Примеры JSON лежат в `09_reports.md`.

B.10. Безопасность и риски

Угроза	Вероятность, ущерб	Мера	Остаточный риск
U-1. Подделка QR	средняя, высокий	HMAC-SHA256 и секретный ключ	низкий
U-2. Повторное использование QR	средняя, средний	nonce и TTL 24 ч	низкий
U-3. Кража реквизитов официанта	низкая, высокий	Только маска, реквизиты у провайдера выплат	низкий
U-4. Двойное списание при повторе	средняя, высокий	`idempotency_key`, одна строка на платёж	низкий

Угроза	Вероятность, ущерб	Мера	Остаточный риск
U-5. Манипуляции официанта с суммой	средняя, средний	Привязка к `receipt_id`, клиент видит сумму до оплаты	средний: сумма не входит в подпись
U-6. Отсутствие аудита	низкая, высокий	Журнал каждого перехода и отказа QR	средний: CSV можно изменить

Данные: PAN и CVV не хранятся никогда, карта вводится на стороне шлюза (PCI DSS); полные реквизиты официанта лежат у провайдера выплат, в логах только `payment_details=****1001`. Секреты: HMAC-ключ и пароль БД лежат в `os.environ`, в боевой версии в менеджере секретов, ротация раз в 90 дней и при утечке. Исключение прототипа: без переменной он берёт учебный ключ из шаблона, им подписаны QR в данных.

Ограничения решения

- Оплата эмулируется: банка, карт и денег нет, данные синтетические.
- HTTP-сервера нет, API описан в `05_api_schemas.yaml`; CSV не защищён от параллельной записи, нагрузку не испытывали.
- Сумма чека приходит в запросе. В боевой версии сервер берёт её из кассы по `receipt_id`.
- Подпись в запросе пока необязательна, чтобы работал формат `qr.json` из шаблона. В API она обязательна.
- Лимит неверных QR описан, но не реализован; картинку QR прототип не рисует, только ссылку.

Как масштабировать: перенести данные из CSV в БД с `UNIQUE` на ключ повтора, подключить настоящий шлюз вместо эмулятора, вынести журнал в отдельное хранилище только на запись. Для Юго-Восточной Азии понадобятся другие валюты, способы оплаты и требования регуляторов, их мы не прорабатывали.

C. Журнал промптов

Дата	Цель	Промпт (сжато)	Результат	Проверка	Использовано

D. Матрица вклада

Участник	Файлы / задачи	Часы	Подпись
Несмачный Егор Дмитриевич, аналитик	`02_roles.md`, `04_requirements.md`, `03_тесты.md`, `09_reports.md`, `08_errors.md`, вопросы по слайдам 2, 6 и 7	[часы]	
Кожаев Тимур Владимирович, архитектор	`03_data_dictionary.md`, `08_ER.md`, `05_api_schemas.yaml`, `06_qr_and_tip_dialog.md`, `02_правила.md`, схемы, вопросы по слайдам 3 и 4	[часы]	
Дурнов Егор Кириллович, разработчик	`04_прототип.py`, `tests_t3.py`, `tests_mutation.py`, `07_payment_emulator_template.md`, вопросы по слайду 5	[часы]	
Солнышкин Леонид Андреевич, презентатор	Слайды, `05_презентация.md`, `11_ИБ.md`, выступление, вопросы по слайдам 1 и 8	[часы]	

E. Артефакты

Git-ветка: team_nachai_case_3. Последний коммит: [скриншот последнего коммита]

Вывод прототипа на qr_001 и qr_003, md5 файла merchants.csv:

```
$ python3 04_прототип.py qr_001.json
{
  "http": 200,
  "payment_id": "p_0101",
  "status": "SUCCESS",
  "amount_total": 668.85,
  "tip_amount": 31.85,
  "confirm_token": "1f9f6cabcdb89285",
  "error_code": null,
  "gateway_code": 0,
  "control_qr": true
}

$ md5sum merchants.csv
006d955099db8bbd8c3286d12f9077f9  merchants.csv
```

```
$ python3 04_прототип.py qr_003.json
{
  "http": 402,
  "payment_id": "p_0102",
  "status": "DECLINED",
  "amount_total": 1047.65,
  "tip_amount": 136.65,
  "confirm_token": "0ccc2c50917aeb9d",
  "error_code": "CARD_DECLINED",
  "gateway_code": 8,
  "control_qr": true,
  "retryable": true,
  "message": "Оплата не прошла, попробуйте другой способ"
}
```

F. Чек-лист приложения

Готово: схема системы из 6 блоков с подписанными потоками, схема состояний платежа, ER-модель, правила Q-1…Q-6 и P-1…P-4 с обоснованием, матрица «ситуация → код → HTTP → retryable», тесты T3-1…T3-5 с фактом.

Команда дополняет до защиты: часы и подписи в матрице вклада, журнал промптов, скриншот последнего коммита, экспорт в PDF.